

Activity 11 — Boeing 737 MAX — Systems Thinking and the Cost of Local Optimisation

Course	Fast-Track Innovations with Agile Design Thinking and Generative AI (GenAI)
WSQ Course Code	TGS-2024049781
Topic	Topic 04: Scaling and Sustaining Innovations with Agile Design Thinking and Generative AI
Duration	50 minutes · Group size: 3–5 participants
Tools	Design Thinking Studio, Padlet Classroom Board, ChatGPT or Microsoft Copilot

Learning Outcome Addressed

LO4 · K2 · K4 — Systems thinking, feedback loops and causality; concept and principles of resource management.

The Real Case

Boeing needed a competitive response to the Airbus A320neo quickly and cheaply. Fitting larger, more efficient engines to the existing 737 airframe changed the aircraft's pitch behaviour, so Boeing added MCAS, a software system that automatically pushed the nose down. Each decision was locally rational: reuse the airframe to save cost and time; solve the aerodynamic consequence in software; keep the aircraft under the same type rating so airlines would not need expensive pilot simulator training; therefore minimise what pilots were told about MCAS. Two crashes killed 346 people and the fleet was grounded worldwide for around 20 months. No single decision was insane in isolation. The system of decisions — commercial pressure, a reused airframe, a software patch, and a training omission driven by the commercial goal — was lethal. This is what a reinforcing feedback loop with no effective balancing loop looks like in practice.

Your Scenario

Your organisation is under pressure to ship an AI-assisted feature before a competitor. Each team is optimising its own deliverable. You have been asked to review the programme for systemic risk.

What You Will Do

Learners map a real systemic failure using interconnectedness, feedback loops and causality, then apply the same mapping to an AI-driven innovation programme to identify where local optimisation is producing global risk.

What you will produce

A causal loop map of the 737 MAX decision chain, and a systemic risk register for the learner's own AI-assisted programme.

Step-by-Step Instructions

1. Open the Padlet classroom board, Activity 11 section.
2. Read the 737 MAX case in the Learner Guide (Activity 11).
3. On paper or in the workspace, draw the decision chain as a loop: each node is a decision, each arrow is 'made necessary'.
4. Mark on your loop the single point where a balancing mechanism would have had the most leverage.
5. Post a photo or description of your loop to Padlet.
6. Now map YOUR AI-assisted programme the same way. Identify one reinforcing loop with no balancing counterpart.
7. Ask ChatGPT to argue why your proposed balancing mechanism would be resisted, and prepare your response.
8. Post your systemic risk register entry: the loop, the leverage point, the mechanism, and who has authority to trigger it.

Discussion Questions

1. Trace the chain: which decision made the next one seem necessary? Where could one balancing loop have broken it?
2. Every decision was locally rational. What does that tell you about relying on individual good judgement as a safety mechanism?
3. Who in the system had the information to see the whole picture, and what stopped that information from changing the outcome?
4. Apply the same analysis to an AI-driven innovation programme. Where does speed pressure create a comparable blind spot?
5. What is the systems-thinking equivalent of 'the scanner is too loud' — the local fix that hides the real problem?

How You Know You Are Done

Your loop map shows causality rather than a timeline, and your risk register names a mechanism with a named owner who has authority to stop work.

Debrief — What the Room Should Conclude

Systems thinking is not an abstract philosophy; it is the discipline that catches failures no individual decision-maker can see. The 737 MAX chain is a textbook reinforcing loop: commercial urgency drove airframe reuse, which drove the software fix, which drove the training omission, each step justified by the previous one, with no balancing loop strong enough to interrupt it. Note who could have interrupted it — regulators, test pilots,

engineers — and what neutralised them: delegated certification, schedule pressure, and a framing in which raising the concern meant arguing against the commercial case. For AI programmes the parallel is direct. Speed pressure produces a model shipped without evaluation, then a guardrail bolted on, then a monitoring gap because monitoring would slow the launch. The countermeasure is structural, not attitudinal: a named balancing mechanism with the authority to stop the line.